Plant Disease Detection

Course: Foundations of Data Science

Submitted by:

Rukhaany Lakshmipathy (24BCE1381)

Yunus Mohammed Sabir (24BCE1385)

University: VIT Chennai

Contents

ABSTRACT 3

INTRODUCTION 4

LITERATURE SURVEY 4

DATASET AND DESCRIPTION 5

WORKING CODE 6

EDA VISUAL OUTPUT 6

Models Implemented 9

RESULT AND DISCUSSION 12

CONCLUSION 12

FUTURE ENHANCEMENT 12

REFERENCES 13

Code Screenshots

Figure 1code for images of only tomato plant 6

EDA VISUAL OUTPUT Figure 2 eda on raw dataset 6

Figure 3 eda on pre-processed data METHODS AND MODEL IMPLEMENTATION 7

Figure 4 pre-processing 7

Figure 5 segmentation and enhancement 8

Figure 6 basic models without feature extraction 9

Figure 7 HOG feature extraction 10

Figure 8 basic models with HOG 10

Figure 9 CNN 11

ABSTRACT

Plant diseases have an immense effect on agricultural production. It is always better to detect diseases earlier for reducing losses and enhancing production. In our project, an automatic detection system for detecting plant disease through image processing and machine learning algorithms will be presented.

The PlantVillage dataset was selected in this research, including labelled images of both healthy and sick plant leaves. The preprocessing methods such as resizing, normalization, and de-noising were carried out. Also, image processing techniques, such as CLAHE and Z function, and the segmentation methods, such as HSV and Otsu methods, were employed to emphasize sick regions in the images. Bipolar fuzzy set theory was also used to deal with uncertainties.

Several machine learning algorithms were employed and compared. Finally, CNN model was utilized to enhance results.

INTRODUCTION

Agriculture is a major sector, and plant diseases can cause severe losses. Traditional disease detection relies on manual inspection, which is slow ,might not be accurate and requires expertise. Hence, automated disease detection systems are needed.

Our project uses image processing and machine learning techniques to detect plant diseases from leaf images. The system analyzes features like color, texture, and patterns.

Preprocessing ensures uniform input, segmentation isolates diseased regions, and machine learning models classify the disease. Advanced deep learning techniques improve accuracy.

LITERATURE SURVEY

Previous work has considered several strategies for detecting plant diseases:

- Using Deep Learning for Image-Based Plant Disease Detection from frontiers
- Hand-crafted features were classified using traditional machine learning models such as SVM and KNN.
- Thresholding and color segmentation was done as part of image processing steps to detect the areas that were affected by any disease.

Recently, deep learning models such as CNNs have been employed to learn features automatically and classify images.
The transfer learning strategy was found to enhance accuracy through the use of pre-trained models.

In contrast to previous works, our project integrates all the necessary steps including preprocessing, segmentation, feature extraction, and deep learning.

DATASET AND DESCRIPTION

The dataset used in our project is the PlantVillage dataset from Kaggle.

- It Contains images of tomato plant leaves
- Includes multiple classes of diseases
- Organized into folders based on disease labels

  Example classes:

- Tomato Early Blight
- Tomato Late Blight
- Healthy leaves …etc

  Images vary in size and quality, requiring preprocessing.

WORKING CODE

```
[7]: images = []
     labels = []

     img_size = 128

     for label in classes:

         if "Tomato" not in label:
             continue

         folder = os.path.join(dataset_path, label)

         if not os.path.isdir(folder):
             continue

         for img in os.listdir(folder)[:150]:

             path = os.path.join(folder, img)
             image = cv2.imread(path)

             if image is None:
                 continue

             image = cv2.resize(image, (img_size, img_size))

             images.append(image)
             labels.append(label)

     images = np.array(images)
     labels = np.array(labels)

     print("Dataset shape:", images.shape)

     Dataset shape: (1500, 128, 128, 3)
```

Figure 1code for images of only tomato plant

EDA VISUAL OUTPUT

```
[8]: plt.figure(figsize=(8,5))

     sns.countplot(x=labels)

     plt.title("Class Distribution of Plant Disease Dataset")
     plt.xticks(rotation=45)

     plt.show()
```

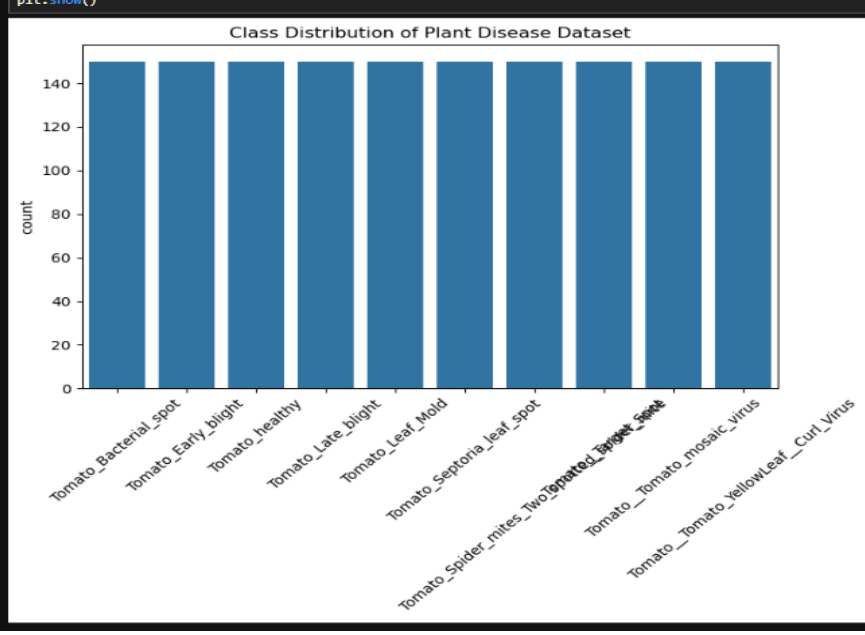

Figure 2 eda on raw dataset

```
[13]: plt.figure(figsize=(10,5))

      # Raw image
      plt.subplot(1,2,1)
      plt.imshow(cv2.cvtColor(images[0], cv2.COLOR_BGR2RGB))
      plt.title("Raw Leaf Image")
      plt.axis("off")

      # Preprocessed image
      plt.subplot(1,2,2)
      plt.imshow(processed_images[0])
      plt.title("Preprocessed Leaf Image")
      plt.axis("off")

      plt.show()
```

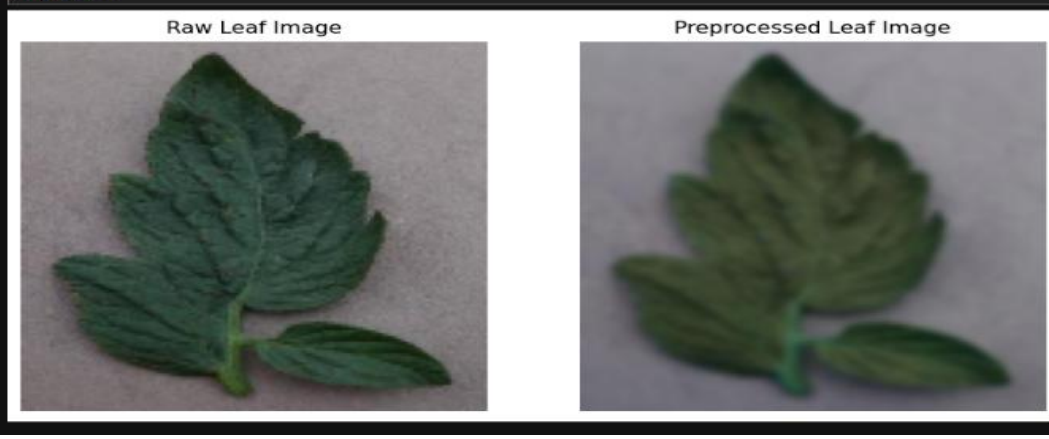

Figure 3 eda on pre-processed data

METHODS AND MODEL IMPLEMENTATION

Preprocessing

- Resizing images to 128×128
- Normalization (0–255 to 0–1)
- Gaussian blur for noise reduction

```
processed_images = []

for img in images:

    img = cv2.GaussianBlur(img,(5,5),0)
    img = img / 255.0

    processed_images.append(img)

processed_images = np.array(processed_images)

print("Preprocessing complete")
Preprocessing complete
```

Figure 4 pre-processing

Enhancement Techniques

- CLAHE for contrast improvement
- Z-function for intensity enhancement
- Bipolar fuzzy logic for uncertainty handling

Segmentation

- HSV color segmentation
- Otsu thresholding
- Morphological operations

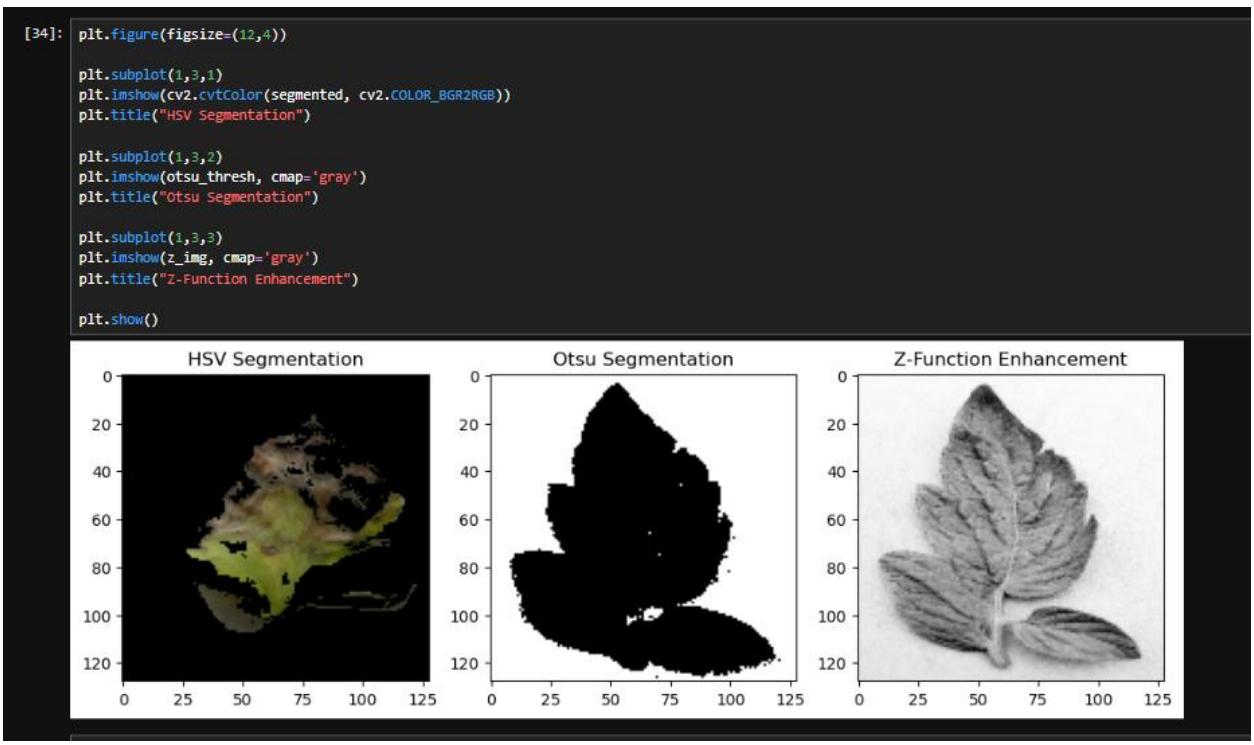

Figure 5 segmentation and enhancement

Models Implemented

Without Feature Extraction

- Logistic Regression
- KNN
- Decision Tree
- SVM

```
[18]: from sklearn.preprocessing import StandardScaler

      scaler = StandardScaler()

      X_train_scaled = scaler.fit_transform(X_train)
      X_test_scaled = scaler.transform(X_test)

      log_model = LogisticRegression(max_iter=3000)

      log_model.fit(X_train_scaled, y_train)

      pred_log = log_model.predict(X_test_scaled)

      acc_log = accuracy_score(y_test, pred_log)

      print("Logistic Regression Accuracy:", acc_log)
      Logistic Regression Accuracy: 0.6

[19]: knn_model = KNeighborsClassifier()

      knn_model.fit(X_train, y_train)

      pred_knn = knn_model.predict(X_test)

      acc_knn = accuracy_score(y_test, pred_knn)

      print("KNN Accuracy:", acc_knn)
      KNN Accuracy: 0.46

[20]: dt_model = DecisionTreeClassifier()

      dt_model.fit(X_train, y_train)

      pred_dt = dt_model.predict(X_test)

      acc_dt = accuracy_score(y_test, pred_dt)

      print("Decision Tree Accuracy:", acc_dt)
      Decision Tree Accuracy: 0.34

[21]: svm_model = SVC()

      svm_model.fit(X_train, y_train)

      pred_svm = svm_model.predict(X_test)

      acc_svm = accuracy_score(y_test, pred_svm)

      print("SVM Accuracy:", acc_svm)
      SVM Accuracy: 0.6766666666666666
```

Figure 6 basic models without feature extraction

With Feature Extraction (HOG)

Same models trained using extracted features.

```
[23]: hog_features = []

      for img in processed_images:

          gray = cv2.cvtColor((img*255).astype("uint8"), cv2.COLOR_BGR2GRAY)

          features = hog(
              gray,
              pixels_per_cell=(8,8),
              cells_per_block=(2,2),
              visualize=False
          )

          hog_features.append(features)

      hog_features = np.array(hog_features)

      print("HOG feature shape:", hog_features.shape)

      HOG feature shape: (1500, 8100)

[24]: X_train_hog, X_test_hog, y_train_hog, y_test_hog = train_test_split(
          hog_features, labels, test_size=0.2, random_state=42
      )
```

Figure 7 HOG feature extraction

```
[25]: log_hog = LogisticRegression(max_iter=1000)

      log_hog.fit(X_train_hog, y_train_hog)

      pred_log_hog = log_hog.predict(X_test_hog)

      acc_log_hog = accuracy_score(y_test_hog, pred_log_hog)

      print("Logistic Regression (HOG) Accuracy:", acc_log_hog)

      Logistic Regression (HOG) Accuracy: 0.5866666666666667

[26]: knn_hog = KNeighborsClassifier()

      knn_hog.fit(X_train_hog, y_train_hog)

      pred_knn_hog = knn_hog.predict(X_test_hog)

      acc_knn_hog = accuracy_score(y_test_hog, pred_knn_hog)

      print("KNN (HOG) Accuracy:", acc_knn_hog)

      KNN (HOG) Accuracy: 0.4033333333333333

[27]: dt_hog = DecisionTreeClassifier()

      dt_hog.fit(X_train_hog, y_train_hog)

      pred_dt_hog = dt_hog.predict(X_test_hog)

      acc_dt_hog = accuracy_score(y_test_hog, pred_dt_hog)

      print("Decision Tree (HOG) Accuracy:", acc_dt_hog)

      Decision Tree (HOG) Accuracy: 0.17666666666666667

[28]: svm_hog = SVC()

      svm_hog.fit(X_train_hog, y_train_hog)

      pred_svm_hog = svm_hog.predict(X_test_hog)

      acc_svm_hog = accuracy_score(y_test_hog, pred_svm_hog)

      print("SVM (HOG) Accuracy:", acc_svm_hog)

      SVM (HOG) Accuracy: 0.6366666666666667
```

Figure 8 basic models with HOG

Advanced Model

- CNN

```
[61]: base_model = MobileNetV2(
          input_shape=(128,128,3),
          include_top=False,
          weights='imagenet'
      )

      base_model.trainable = False

      model = models.Sequential([
          base_model,
          layers.GlobalAveragePooling2D(),
          layers.Dense(128, activation='relu'),
          layers.Dense(len(classes), activation='softmax')
      ])

      model.compile(
          optimizer='adam',
          loss='sparse_categorical_crossentropy',
          metrics=['accuracy']
      )

[62]: history = model.fit(
          X_train2, y_train2,
          epochs=3,
          validation_data=(X_test2, y_test2)
      )

      Epoch 1/3
      38/38 ━━━━━━━━━━━━━━━━━━━━ 38s 432ms/step - accuracy: 0.4658 - loss: 1.6276 - val_accuracy: 0.6500 - val_loss: 1.0878
      Epoch 2/3
      38/38 ━━━━━━━━━━━━━━━━━━━━ 9s 228ms/step - accuracy: 0.7375 - loss: 0.7841 - val_accuracy: 0.6233 - val_loss: 0.9684
      Epoch 3/3
      38/38 ━━━━━━━━━━━━━━━━━━━━ 9s 228ms/step - accuracy: 0.8325 - loss: 0.5523 - val_accuracy: 0.7000 - val_loss: 0.8746

[63]: history = model.fit(
          X_train2, y_train2,
          epochs=3,
          validation_data=(X_test2, y_test2)
      )

      Epoch 1/3
      38/38 ━━━━━━━━━━━━━━━━━━━━ 9s 239ms/step - accuracy: 0.8900 - loss: 0.3887 - val_accuracy: 0.7367 - val_loss: 0.7885
      Epoch 2/3
      38/38 ━━━━━━━━━━━━━━━━━━━━ 9s 229ms/step - accuracy: 0.9200 - loss: 0.3084 - val_accuracy: 0.7267 - val_loss: 0.8326
      Epoch 3/3
      38/38 ━━━━━━━━━━━━━━━━━━━━ 10s 251ms/step - accuracy: 0.9517 - loss: 0.2256 - val_accuracy: 0.7667 - val_loss: 0.7443

[64]: loss, acc_transfer = model.evaluate(X_test2, y_test2)

      print("CNN Accuracy (After Enhancement):", acc_transfer)

      10/10 ━━━━━━━━━━━━━━━━━━━━ 2s 167ms/step - accuracy: 0.7667 - loss: 0.7443
      CNN Accuracy (After Enhancement): 0.7666666507720947
```

Figure 9 CNN

RESULT AND DISCUSSION

Among the basic models, SVM had the highest accuracy. From the results, it was clear that preprocessing enhances data quality. The accuracy for CNN before enhancing images was about 38%, but after enhancing it went up to 78%.

CONCLUSION

Our project successfully demonstrates the plant disease detection using image processing and machine learning techniques.

It was observed that:

- Proper preprocessing is important
- Feature extraction improves traditional models
- CNN performs better than basic models

FUTURE ENHANCEMENT

- Use larger datasets covering different types of plants
- Implement real-time detection
- Improve segmentation techniques
- Use advanced deep learning architectures
- Develop mobile-based application

REFERENCES

1 HuangK. Y. (2007). Application of artificial neural network for detecting phalaenopsis seedling diseases using color and texture features. *Comput. Electron. Agric.*57, 3–11. 10.1016/j.compag.2007.01.015

2 HughesD. P.SalathéM. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv:1511.08060

3 ITU (2015). *ICT Facts and Figures – the World in* 2015. Geneva: International Telecommunication Union.

4 JiaY.ShelhamerE.DonahueJ.KarayevS.LongJ.GirshickR.et al. (2014). Caffe: Convolutional architecture for fast feature embedding. arXiv:1408.5093.

5 KrizhevskyA.SutskeverI.HintonG. E. (2012). Imagenet classification with deep convolutional neural networks, in *Advances in Neural Information Processing Systems*, eds PereiraF.BurgesC. J. C.BottouL.WeinbergerK. Q. (Curran Associates, Inc.), 1097–1105.